Worksheet 5

Student Name: Anish Chand **UID: 25MCI10096**
Branch: MCA (AI & ML) **Section/Group: 25MAM-4 (B)**
Semester: 2nd **Date of Performance: 02-04-2026**
Subject Name: Advanced Internet Programming **Subject Code: 25CAP-656**

1. Aim/Overview of the practical:
Implement CRUD (Create, Read, Update, Delete) operations in NodeJS using MongoDB and MySQL.

2. Apparatus:
Supported Operating Systems: Windows 11 64-bit Pro
Programming Language: HTML and NodeJS
Development Environment: VS Code

3. Coding: -
Index.js

```
const express = require("express");
const mongoose = require("mongoose");
const bodyParser = require("body-parser");
const session = require("express-session");

const app = express();

app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static("public"));
app.set("view engine", "ejs");

app.use(session({
  secret: "secret123",
  resave: false,
  saveUninitialized: false
}));

mongoose.connect("mongodb://127.0.0.1:27017/employeeDB")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log("DB Error:", err));

const empSchema = new mongoose.Schema({
```

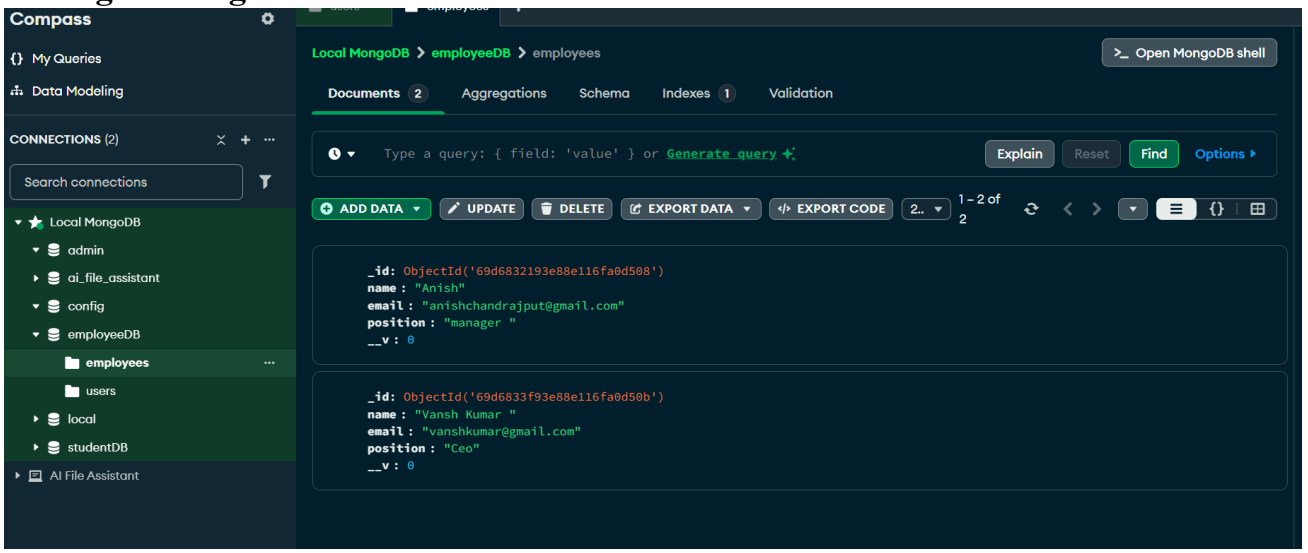
```
  name: String,
  email: String,
  position: String
});
const Employee = mongoose.model("Employee", empSchema);

const userSchema = new mongoose.Schema({
  username: String,
  password: String
});
const User = mongoose.model("User", userSchema);

app.get("/", (req, res) => {
  res.render("login");
});

app.get("/register", (req, res) => {
  res.render("register");
});

app.post("/register", async (req, res) => {
  try {
    await User.create(req.body);
    res.redirect("/");
  } catch (err) {
    res.send("Error in registration");
  }
});

app.post("/login", async (req, res) => {
  const { username, password } = req.body;

  try {
    const user = await User.findOne({ username, password });

    if (user) {
      req.session.user = username;
      res.redirect("/dashboard");
    } else {
      res.send("Invalid Login");
    }
  } catch (err) {
    res.send("Login Error");
  }
```

```
});

app.get("/dashboard", async (req, res) => {
  if (!req.session.user) return res.redirect("/");

  try {
    const data = await Employee.find();
    res.render("dashboard", { employees: data });
  } catch (err) {
    res.send("Error loading dashboard");
  }
});

app.get("/add", (req, res) => {
  if (!req.session.user) return res.redirect("/");
  res.render("add");
});

app.post("/add", async (req, res) => {
  try {
    await Employee.create(req.body);
    res.redirect("/dashboard");
  } catch (err) {
    res.send("Error adding employee");
  }
});

app.get("/delete/:id", async (req, res) => {
  try {
    await Employee.findByIdAndDelete(req.params.id);
    res.redirect("/dashboard");
  } catch (err) {
    res.send("Delete error");
  }
});

app.get("/edit/:id", async (req, res) => {
  try {
    const data = await Employee.findById(req.params.id);
    res.render("edit", { emp: data });
  } catch (err) {
    res.send("Error loading edit page");
  }
```

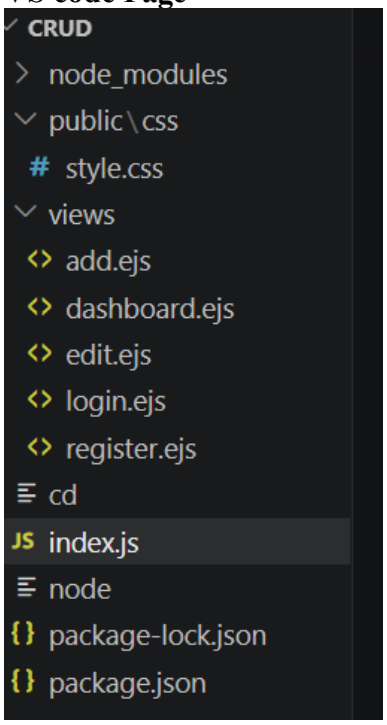
```
});

app.post("/edit/:id", async (req, res) => {
  try {
    await Employee.findByIdAndUpdate(req.params.id, req.body);
    res.redirect("/dashboard");
  } catch (err) {
    res.send("Update error");
  }
});

app.get("/logout", (req, res) => {
  req.session.destroy(() => {
    res.redirect("/");
  });
});

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Output:

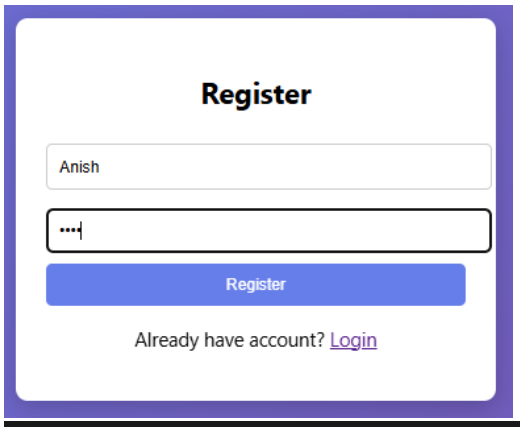
MongoDB Page

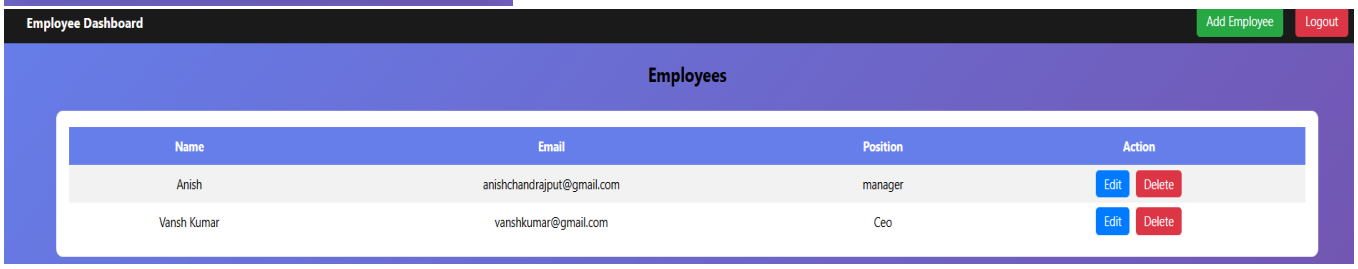


VS code Page



Register Page





4. Learning outcomes (What I have learnt):

- Understand how to build a web application using **NodeJS and Express framework**.
- Learn how to connect a **MongoDB database** with a NodeJS application using **Mongoose**.
- Implement **CRUD operations (Create, Read, Update, Delete)** to manage user data.
- Develop a simple **user interface using HTML and CSS** to interact with the backend.

Gain practical experience in handling **server-side programming and database management**.